GPU ab-initio gene finders on Pelle and Arrhenius

Guilherme Dias · NBIS

2026-10-21

Getting allocations

- Pelle
  - Approved in 1 day, no questions asked

Getting allocations

- Arrhenius
  - Basically same proposal I submitted to Pelle
- submitted Sep 22nd
- comment from NAISS Sep 29th:

> You need to update the resource usage to explain why you need 200 GPU hours. How many hours per run, how many run per months and so on.
>
> The number you asked for needs to be justified.

- approved Oct 2nd

What we tested

- **4 gene finders:** Tiberius, OrionGeno, Vipsania (plain and finetune), Helixer
- **1 genome:** the whole human genome (3.1 Gb), one job per tool and hardware
- **2 clusters, 6 kinds of hardware**

cluster	GPU nodes	CPU nodes
Pelle (UPPMAX)	NVIDIA T4, L40S, H100 · x86 host	AMD Zen 4, 24 cores
Arrhenius (NAISS)	NVIDIA GH200 · **Arm** host	AMD Turin, 24 cores

- We measured wall time, GPU use and energy, memory, and the number of genes found

> **Bottom line:** a like-for-like speed test of GPU vs CPU, and of GPU vs GPU.

Arrhenius is complete; Pelle has 5 gaps, all Vipsania

Vipsania counts as two tools (plain and finetune). OrionGeno and Vipsania-finetune are GPU-only.

Pelle	T4	L40S	H100	CPU
Tiberius	done	done	done	done
Helixer	done	done	done	done
OrionGeno	skipped (GPU too old)	done	done	GPU-only
Vipsania-plain	**failed** (out of GPU memory)	done	done	**no result** (likely RAM)
Vipsania-finetune	**not run**	**not run**	**not run**	GPU-only

- **Pelle:** 12 of 17 runs done (5 tools × 4 hardware = 20, minus 2 GPU-only on CPU, minus OrionGeno on T4)
- **Arrhenius:** 8 of 8 done (5 tools on GH200 + 3 on CPU)
- The Vipsania CPU job on Arrhenius needed 187 GB of RAM, more than the 180 GB requested

> **Bottom line:** enough results to compare. To finish Pelle: rerun Vipsania-plain on CPU with more memory and run Vipsania-finetune.

A modern GPU annotates the human genome in about 1 hour

> **Bottom line:** Tiberius, OrionGeno and Vipsania take about 1 h on H100 or GH200, versus 6–60 h on CPU. Helixer is the slow one (9–22 h).

The GPU speed-up depends heavily on the tool

tool	GH200	CPU (24 cores)	speed-up
Vipsania-plain	55 min	60.6 h	**67×**
Tiberius	54 min	6.3 h	**7×** (10× on Pelle)
Helixer	9.2 h	14.5 h	**1.6×** (2× on Pelle)
OrionGeno, Vipsania-finetune	51 min, 2.5 h	–	GPU only

- Helixer keeps the GPU busy only about 11 % of the time: most of its work runs on the CPU

> **Bottom line:** a GPU is essential for Vipsania, very useful for Tiberius, and of little benefit for Helixer.

Which GPU?

- **GH200** (Arrhenius) is the fastest: 12–19 % faster than the H100
- **H100** (Pelle) is a close second
- **L40S** takes about twice as long as the H100 for OrionGeno and Vipsania, and about the same for Tiberius and Helixer
- **T4** is only practical for Tiberius (5.9 h). Helixer takes 22 h on it; Vipsania and OrionGeno don’t run.

> **Bottom line:** use H100 or GH200 when you can; L40S is fine; avoid T4.

The hardware doesn't change the answer

tool	genes found (all hardware)	vs Ensembl
Tiberius	22,111 – 22,117	1.10
OrionGeno	19,445 – 19,453	0.97
Vipsania-plain	22,995 – 23,001	1.14
Vipsania-finetune	18,808	0.93
Helixer	21,684 – 21,685; **21,421 on GH200**	1.08

- GPU and CPU runs agree to within a few genes
- Helixer on GH200 finds 1.2 % fewer genes, probably because of the older TensorFlow version in its Arm image
- We measured speed only; accuracy (e.g. BUSCO) has not been tested yet

> **Bottom line:** pick hardware for speed and cost; the predictions stay the same.

Arm GPU nodes are becoming normal in HPC

- NVIDIA's GH200 puts an **Arm CPU (Grace)** and a Hopper GPU on one board. Its successors (GB200, GB300) keep the Arm CPU.
- Several of Europe's biggest new systems use it: **JUPITER** (Germany, Europe's first exascale machine), **Alps** (Switzerland), **Isambard-AI** (UK) and **Arrhenius** (Sweden)
- Clusters are often mixed: Arrhenius has Arm GPU nodes, but x86 CPU nodes and an x86 login node
- The hardware is mainstream, but scientific software is not there yet: **none of the four gene finders publishes an Arm version**

> **Bottom line:** expect more Arm GPU systems. Bioinformatics tools need Arm builds to use them.

Developer vs NGC container images

- **Developer image:** the container published by the tool's own developers (Docker Hub)
  - the tool plus the exact TensorFlow/PyTorch and CUDA versions they tested
  - x86 only, so it is used as-is on Pelle (except OrionGeno, which we build from its Dockerfile)
- **NGC image:** NVIDIA's ready-made TensorFlow or PyTorch container (`nvcr.io`)
  - the framework and CUDA only, with no tool inside; built for both x86 and Arm
  - the tag is the release date: *25.02* = February 2025, which contains TensorFlow 2.17
- **Our Arm images** = an NGC image with the tool installed on top

	Pelle (x86 images)	**Arrhenius (Arm: NGC + tool)**
Tiberius, Vipsania	TensorFlow 2.17	TensorFlow 2.17
OrionGeno	PyTorch 2.7	PyTorch 2.7
Helixer	TensorFlow 2.15	TensorFlow 2.14

> **Bottom line:** the same tools on nearly the same software; Helixer is the one exception.

Rebuilding for Arm took the most effort

- **pip quietly swapped NVIDIA's GPU TensorFlow for a CPU-only one.** We now lock NVIDIA's packages and make the build fail if they change.
- **Helixer:** newer NVIDIA images (cuDNN 9) break its neural network layers, so it uses an older image (TensorFlow 2.14)
- **OrionGeno:** its GPU kernels don't exist for Arm, so we compile them ourselves
- **Builds must run on an Arm GPU node** with at least 64 GB of memory. On the x86 login node they silently produce an x86 image.
- **GPU problems only show up at run time**, so each fix needs another test run

> **Bottom line:** budget real time for containers when moving to Arm GPU nodes.

Pelle and Arrhenius had different problems

	Pelle (x86 + T4 / L40S / H100)	Arrhenius (Arm + GH200)
containers	upstream images used as published (OrionGeno built from its Dockerfile)	all four rebuilt from NGC images
hardware	T4 too small or too old for some tools; 3 broken GPU nodes (now excluded)	none
software	GPU monitoring (`nvidia-smi`) failing on some nodes	wrong CUDA driver library picked up; user `~/.bashrc` leaking into containers
accounting	one project	separate GPU project and partition

> **Bottom line:** on Pelle the problems were hardware; on Arrhenius they were software.

Take-home

1. **About 1 hour** to annotate the human genome on a modern GPU, for Tiberius, OrionGeno and Vipsania
2. **GPU benefit varies by tool:** 67× for Vipsania, 7–10× for Tiberius, only 2× for Helixer
3. **GH200 > H100 > L40S ≫ T4**
4. **Same genes on every kind of hardware**
5. **Arm GPU nodes are the future, but tools aren’t ready:** porting containers is the hidden cost

Next steps

- Fill the Pelle gaps: Vipsania on CPU with more memory, and Vipsania-finetune on H100/L40S
- Test accuracy, not just speed (BUSCO, comparison with Ensembl)
- Repeat runs to measure variation; pin the container versions
- Report the broken Pelle GPU nodes to UPPMAX

Sources for the Arm slide: JUPITER · Alps · Arrhenius